

Customer Churn

CUSTOMER CHURN PREDICTION PROJECT

Understanding why customers leave is crucial for any business. This project delves into customer behavior to identify patterns leading to churn, enabling proactive retention strategies.



What is Customer Churn?



Definition

Customer churn refers to the phenomenon where customers stop using a company's product or service, often switching to a competitor.



Impact

High churn rates can significantly impact revenue, growth, and brand reputation. Retaining existing customers is often more cost-effective than acquiring new ones.



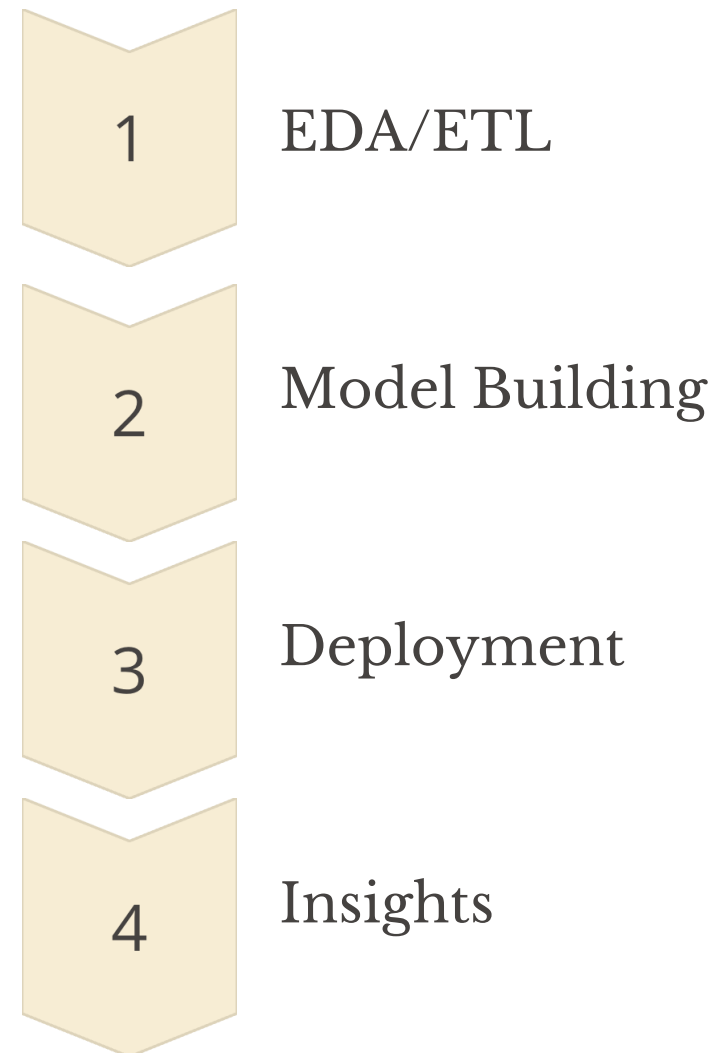
Prediction

By analyzing historical data, we can predict which customers are at risk of churning, allowing businesses to intervene before it's too late.



Project Workflow: From Data to Actionable Insights

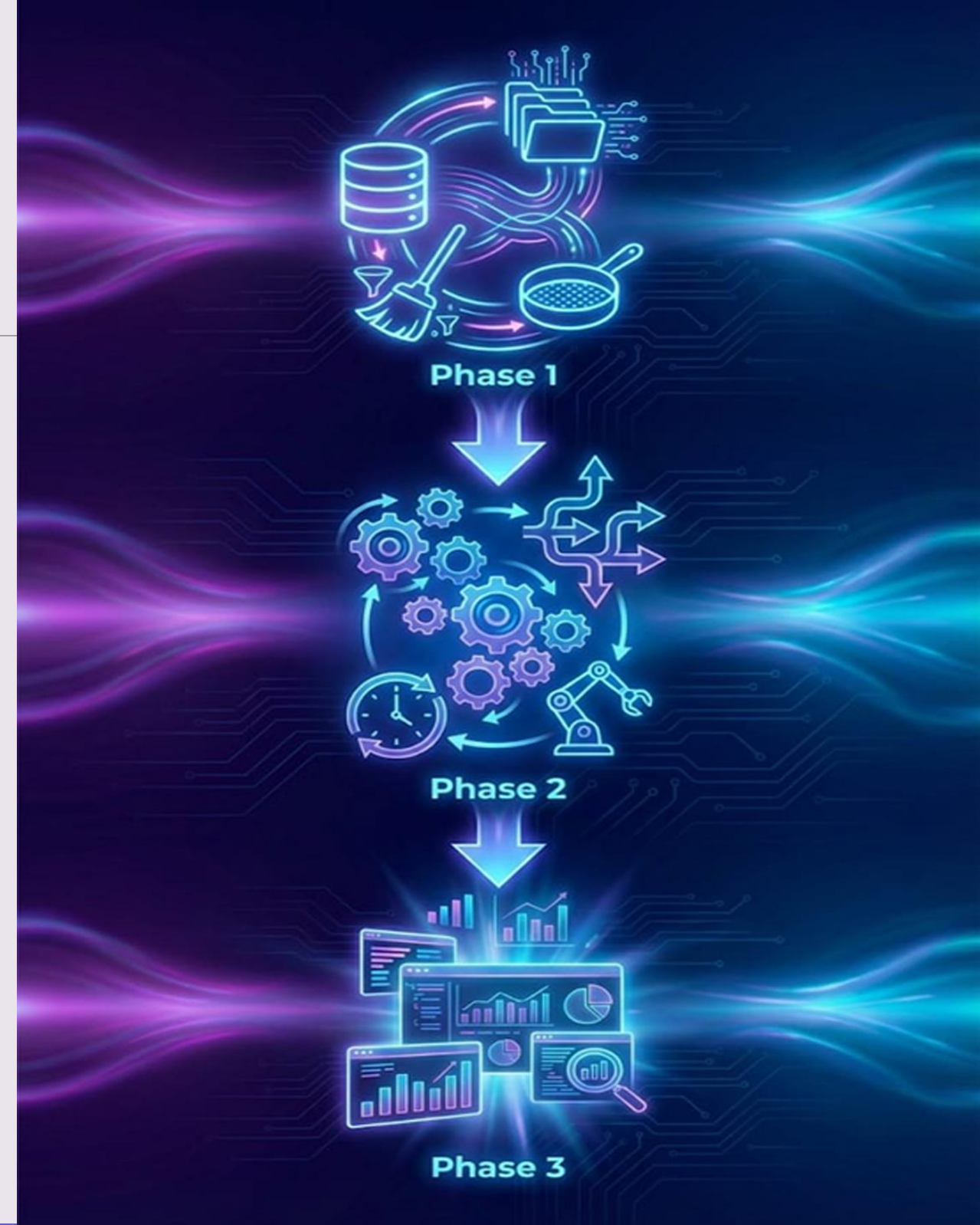
Our project follows a structured approach, transforming raw customer data into predictive models and actionable business intelligence.



Three Phases of Our Pipeline

- 1 **Phase 1: Data Preparation**
Extract, Load & Clean Data for Analysis
- 2 **Phase 2: Orchestration**
Pipeline Orchestration & Scheduling
- 3 **Phase 3: Visualization**
Insights & Visualization with Power BI

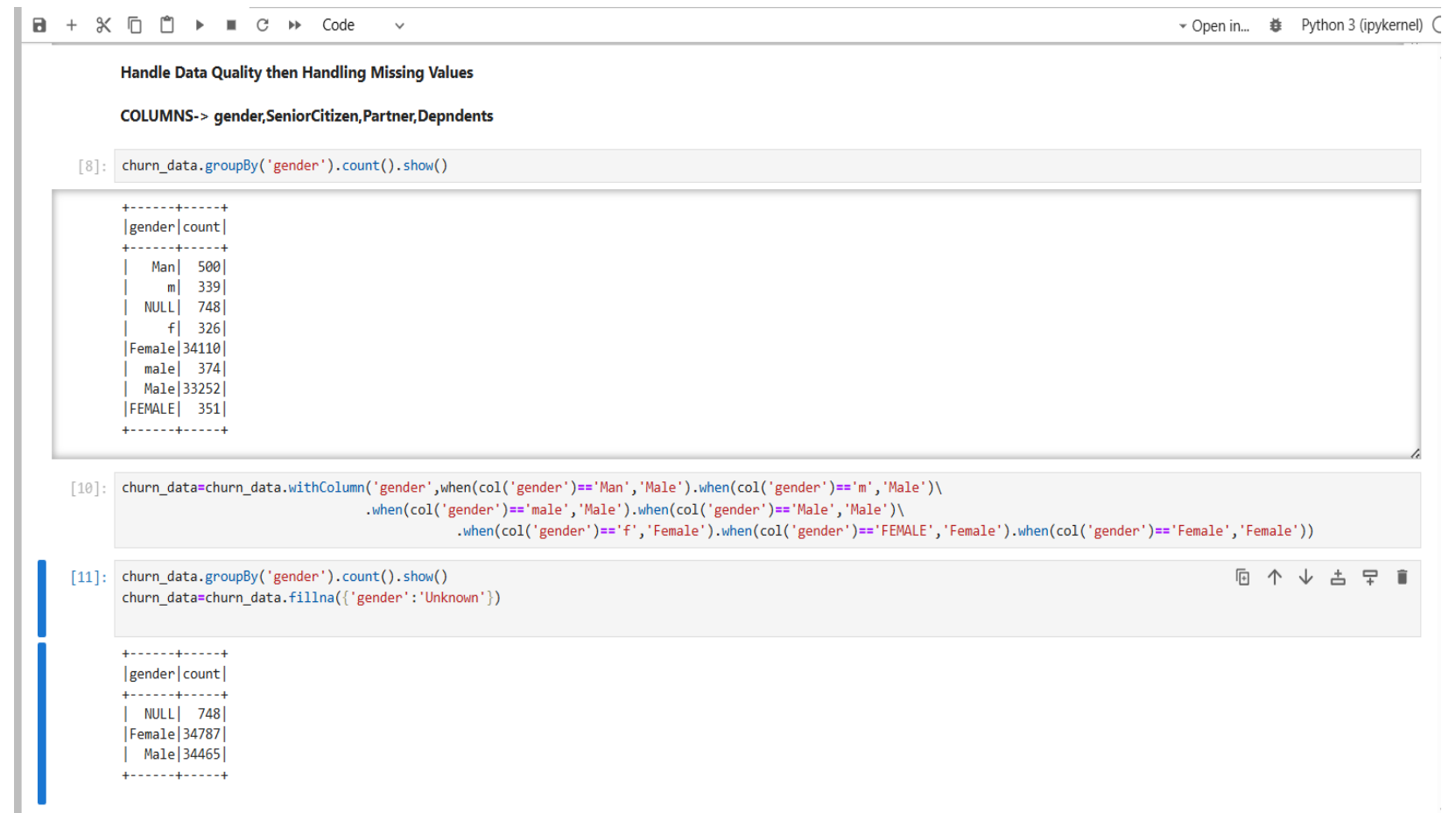
Each phase was meticulously executed, leveraging specialized skills within our team to ensure robust and reliable results.



Phase 1: Data Preparation

Standardizing Categorical Values and Handling Missing Gender Data

This section shows the process of cleaning and unifying inconsistent gender entries in the dataset. Variations such as “Man,” “m,” and different spellings of “Female” were standardized into clear categories. Remaining missing values were then filled with “Unknown” to ensure a clean and reliable dataset for analysis



The screenshot shows a Jupyter Notebook interface with the following content:

Handle Data Quality then Handling Missing Values

COLUMNS-> gender,SeniorCitizen,Partner,Depndents

```
[8]: churn_data.groupby('gender').count().show()
```

gender	count
Man	500
m	339
NULL	748
f	326
Female	34110
male	374
Male	33252
FEMALE	351

```
[10]: churn_data=churn_data.withColumn('gender',when(col('gender')== 'Man', 'Male').when(col('gender')== 'm', 'Male')\
    .when(col('gender')== 'male', 'Male').when(col('gender')== 'Male', 'Male')\
    .when(col('gender')== 'f', 'Female').when(col('gender')== 'FEMALE', 'Female').when(col('gender')== 'Female', 'Female'))
```

```
[11]: churn_data.groupby('gender').count().show()
churn_data=churn_data.fillna({'gender': 'Unknown'})
```

gender	count
NULL	748
Female	34787
Male	34465

Missing Values Check and Data Preprocessing

The image shows a key step in the data preprocessing phase of the Customer Churn analysis project. In this stage, negative values in numerical columns like *tenure* and *MonthlyCharges* are corrected, and the PySpark **Imputer** is used to fill missing values using the median strategy. A new feature, **TotalCharges**, is then created by multiplying tenure by the monthly charges. The code also includes displaying the count of missing values for each column to ensure the dataset is clean and ready for further analysis or model building.

```
Customer Churn Analysis.ipynb
+
[52]: from pyspark.ml.feature import Imputer
      ## TO GET THE MEDIAN VALUE OF THE DATA TO FILL IT WITH THE MISSING VALUE
      ## Found negative values

[53]: churn_data=churn_data.withColumn('tenure',abs(col('tenure'))).withColumn('MonthlyCharges',abs(col('MonthlyCharges')))

[54]: imputer=Imputer(inputCols=['tenure','MonthlyCharges'],outputCols=['tenure','MonthlyCharges']).setStrategy('median')
      churn_data=imputer.fit(churn_data).transform(churn_data)

[55]: churn_data=churn_data.withColumn('TotalCharges',(col('tenure')*col('MonthlyCharges')))

[56]: null_data4=churn_data.select([count(when(col(c).isNull(),1)).alias(c) for c in churn_data.columns])
      null_data4.select(null_data4.columns[0:5]).show()
      null_data4.select(null_data4.columns[5:10]).show()
      null_data4.select(null_data4.columns[10:15]).show()
      null_data4.select(null_data4.columns[15:25]).show()

+-----+-----+-----+-----+-----+
|tenure|PhoneService|MultipleLines|InternetService|OnlineSecurity|
+-----+-----+-----+-----+-----+
|      0|            0|            0|            0|            0|
+-----+-----+-----+-----+-----+

+-----+-----+-----+-----+-----+
|OnlineBackup|DeviceProtection|TechSupport|StreamingTV|StreamingMovies|
+-----+-----+-----+-----+-----+
|            0|            0|            0|            0|            0|
+-----+-----+-----+-----+-----+

+-----+-----+-----+-----+-----+-----+
|Contract|PaperlessBilling|PaymentMethod|MonthlyCharges|TotalCharges|Churn|
+-----+-----+-----+-----+-----+-----+
|            0|            0|            0|            0|            0|      0|
+-----+-----+-----+-----+-----+-----+
```


Phase 2: Pipeline Orchestration & Scheduling with Airflow

Exploring Customer Churn Data

This image shows the initial data view of the churn_data table in SQL Server, displaying key customer details like **gender**, **tenure**, and various **service subscriptions**. This foundational inspection is essential for understanding the dataset's structure before proceeding with advanced data analysis for churn prediction.

SQLQuery1.sql - A...STRATOR\Min (56))

select * from churn_data

146 %

Results Messages

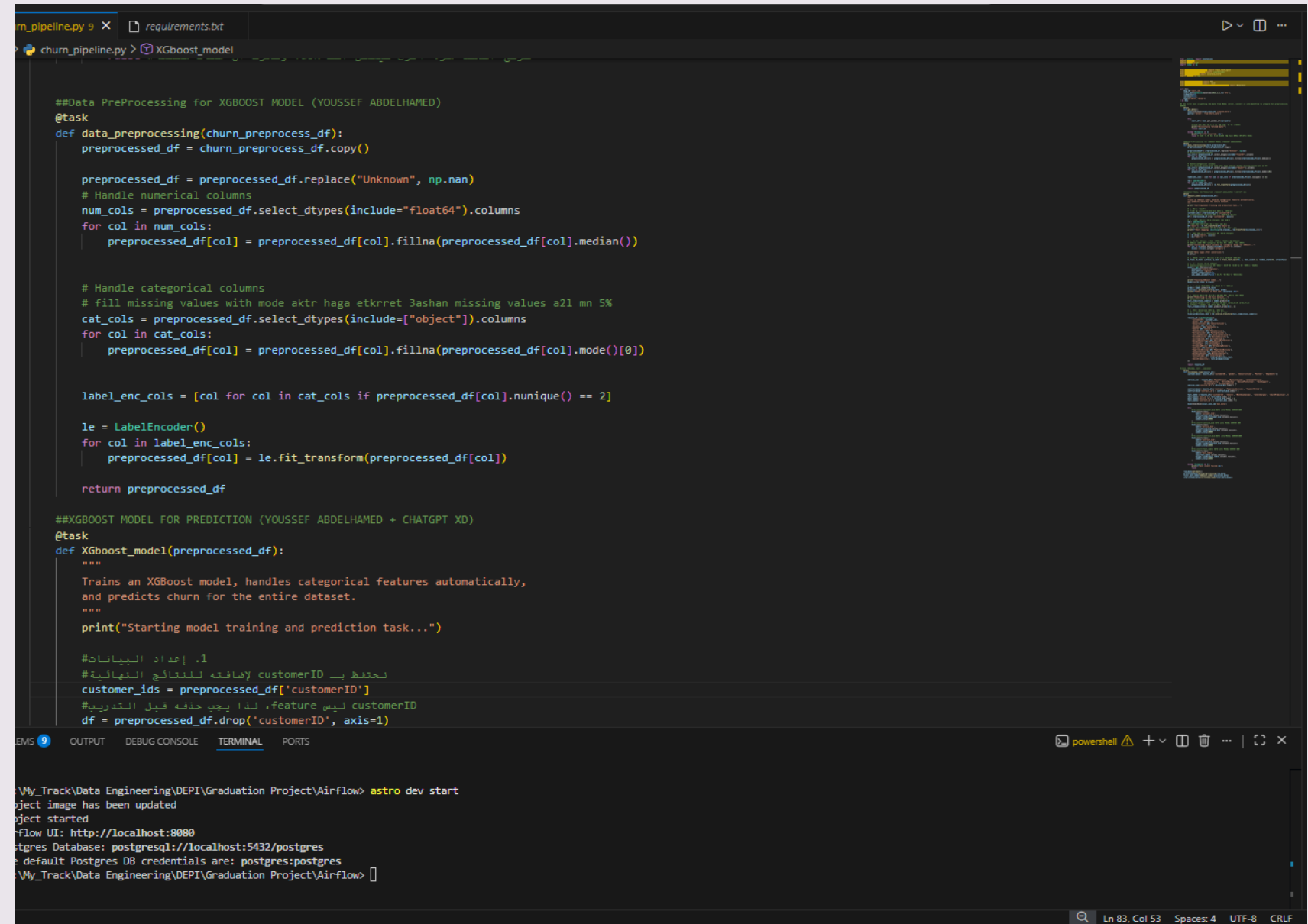
	customerID	gender	SeniorCitizen	Partner	Depndents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupp
1	CUST00001	Male	No	No	Yes	3	Yes	Yes	No	No	No	No	No
2	CUST00002	Male	Yes	Yes	No	2	Yes	Yes	DSL	No	No	No	Yes
3	CUST00003	Female	No	No	No	42	Yes	Yes	DSL	No	Yes	No	Unknown
4	CUST00004	Female	No	No	Yes	40	Yes	Yes	Fiber optic	No	No	Yes	No
5	CUST00005	Male	Yes	Yes	Yes	17	Yes	Unknown	Fiber optic	Yes	No	Yes	No
6	CUST00006	Male	No	Yes	No	18	Yes	No	Fiber optic	No	No	Unknown	Yes
7	CUST00007	Female	No	No	No	25	Yes	Yes	No	No	No	No	No
8	CUST00008	Female	No	Yes	No	33	Yes	Yes	Fiber optic	No	Unknown	No	No
9	CUST00009	Female	No	Yes	No	6	Yes	Yes	DSL	Yes	No	No	No
10	CUST00010	Female	No	Yes	No	5	Yes	No	No	No	No	No	No
11	CUST00011	Female	No	No	No	18	Yes	No	No	No	No	No	No
12	CUST00012	Male	No	Yes	No	22	Yes	No	No	No	No	No	No
13	CUST00013	Female	Yes	Yes	No	10	Yes	Yes	DSL	No	No	No	Yes
14	CUST00014	Male	No	Yes	No	2	Yes	Unknown	Fiber optic	No	No	No	Unknown
15	CUST00015	Male	No	Yes	No	8	Yes	Yes	Fiber optic	Yes	No	Yes	No
16	CUST00016	Female	No	No	No	5	No	No	Fiber optic	Yes	No	Yes	No

Query executed successfully. ADMINISTRATOR\SQLEXPRESS01 ... ADMINISTRATOR\Min (56) Churn_Project 00:00:00 70,000 rows

Ln 2 Col 1 Ch 1 INS

Data Preprocessing for Customer Churn Prediction

This Python script in VS Code demonstrates the initial steps of a machine learning workflow, including **handling missing data** and **preprocessing features** before training an **XGBoost model** for customer churn prediction.



```
##Data PreProcessing for XGBOOST MODEL (YOUSSEF ABDELHAMED)
@task
def data_preprocessing(churn_preprocess_df):
    preprocessed_df = churn_preprocess_df.copy()

    preprocessed_df = preprocessed_df.replace("Unknown", np.nan)
    # Handle numerical columns
    num_cols = preprocessed_df.select_dtypes(include="float64").columns
    for col in num_cols:
        preprocessed_df[col] = preprocessed_df[col].fillna(preprocessed_df[col].median())

    # Handle categorical columns
    # fill missing values with mode aktr haga etkrret 3ashan missing values a2l mn 5%
    cat_cols = preprocessed_df.select_dtypes(include=["object"]).columns
    for col in cat_cols:
        preprocessed_df[col] = preprocessed_df[col].fillna(preprocessed_df[col].mode()[0])

    label_enc_cols = [col for col in cat_cols if preprocessed_df[col].nunique() == 2]

    le = LabelEncoder()
    for col in label_enc_cols:
        preprocessed_df[col] = le.fit_transform(preprocessed_df[col])

    return preprocessed_df

##XGBOOST MODEL FOR PREDICTION (YOUSSEF ABDELHAMED + CHATGPT XD)
@task
def XGboost_model(preprocessed_df):
    """
    Trains an XGBoost model, handles categorical features automatically,
    and predicts churn for the entire dataset.
    """
    print("Starting model training and prediction task...")

    #1. إعداد البيانات
    # نحفظ بـ customerID لإضافته للنتائج النهائية
    customer_ids = preprocessed_df['customerID']
    # customerID ليس feature، لذا يجب حذفه قبل التدريب
    df = preprocessed_df.drop('customerID', axis=1)
```

terminal

```
~\My_Track\Data Engineering\DEPI\Graduation Project\Airflow> astro dev start
object image has been updated
object started
Airflow UI: http://localhost:8080
postgres Database: postgresql://localhost:5432/postgres
default Postgres DB credentials are: postgres:postgres
~\My_Track\Data Engineering\DEPI\Graduation Project\Airflow>
```


Airflow DAG: Automating Our Churn Pipeline

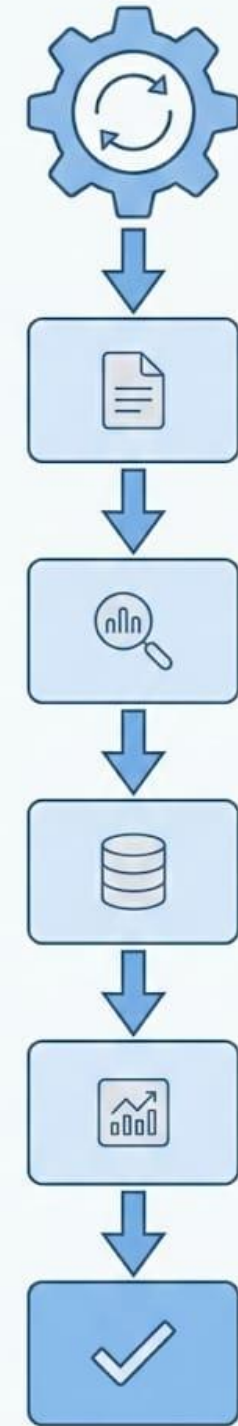
The `Airflow` folder contains our DAG definitions, outlining the sequence and dependencies of tasks in our data pipeline. This code defines how data is processed, models are trained, and results are prepared for visualization.

Code Implementation

The Python scripts within the `Airflow` folder represent individual tasks (e.g., data cleaning, feature engineering, model training) that are orchestrated by Airflow. Each task is defined with clear inputs and outputs, ensuring modularity and reusability.

Benefits of Airflow

Airflow provides robust scheduling, monitoring, and error handling capabilities. It ensures that our churn prediction model is always fed with fresh data and that insights are generated consistently.



```
churn_pipeline.py 9 X requirements.txt
dags > churn_pipeline.py > ...
71
72 ##XGBOOST MODEL FOR PREDICTION
73 @task
74 def XGboost_model(preprocessed_df):
75     """
76     Trains an XGBoost model, handles categorical features automatically,
77     and predicts churn for the entire dataset.
78     """
79     print("Starting model training and prediction task...")
80
81     #1. إعداد البيانات
82     # نحفظ بـ customerID إضافته للنتائج النهائية
83     customer_ids = preprocessed_df['customerID']
84     # customerID ليس feature، لذا يجب حذفه قبل التدريب
85     df = preprocessed_df.drop('customerID', axis=1)
86
87     #2. تحويل المتغير الهدف (Target) إلى أرقام
88     le = LabelEncoder()
89     # تحويل عمود Churn من 'No'/'Yes' إلى 0/1
90     df['Churn'] = le.fit_transform(df['Churn'])
91     # طباعة التحويل للتأكد (مثال: {'No': 0, 'Yes': 1})
92     print(f"Churn mapping: {dict(zip(le.classes_, le.transform(le.classes_)))}")
93
94     #3. فصل المتغيرات (Features) عن الهدف (Target)
95     X = df.drop('Churn', axis=1)
96     y = df['Churn']
97
98     #4. الخطوة السحرية: تحويل الأعمدة الفئوية لـ XGBoost
99     # XGBoost يفهم نوع 'category' مباشرة عند تفعيل خاصية معينة
100     print("Converting object columns to 'category' dtype for XGBoost...")
101     for col in X.select_dtypes(include=['object']).columns:
102         X[col] = X[col].astype('category')
103
104     print("Data types after conversion:")
105     X.info()
106
107     #5. تقسيم البيانات للتدريب والاختبار (لتقييم الموديل)
108     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
109
110     #6. بناء وتدريب موديل XGBoost
111     # enable_categorical=True هو المفتاح للتعامل التلقائي مع الأعمدة الفئوية
112     model = xgb.XGBClassifier(
113         objective='binary:logistic',
```

Advanced XGBoost Modeling for Churn Prediction

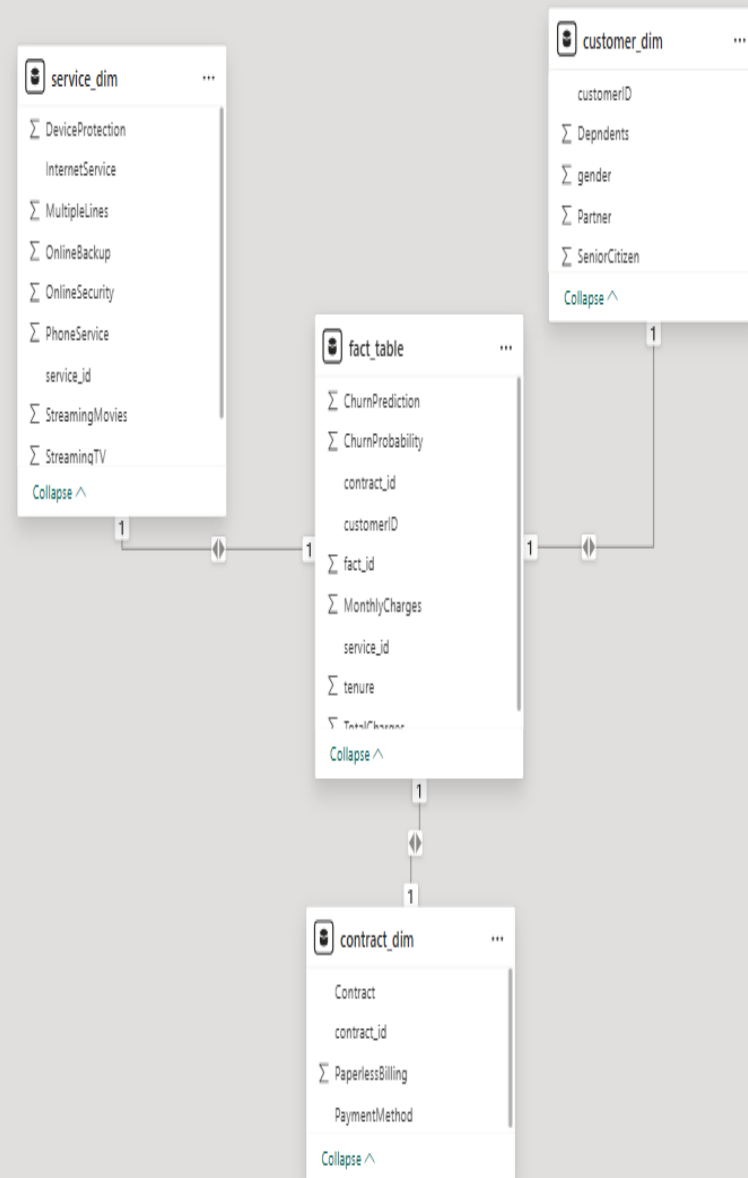
This Python code prepares the dataset for the final prediction stage by **encoding the target variable ('Churn')** and **splitting the data for training and testing**. It then initiates the **XGBoost Classifier model** for building a robust predictive solution.

Implementing Data Modeling: The Star Schema

This screenshot illustrates the conceptual design of our Star Schema for churn analysis. The central fact table would contain key metrics, linked to dimension tables such as customer demographics, service usage, and transaction history.

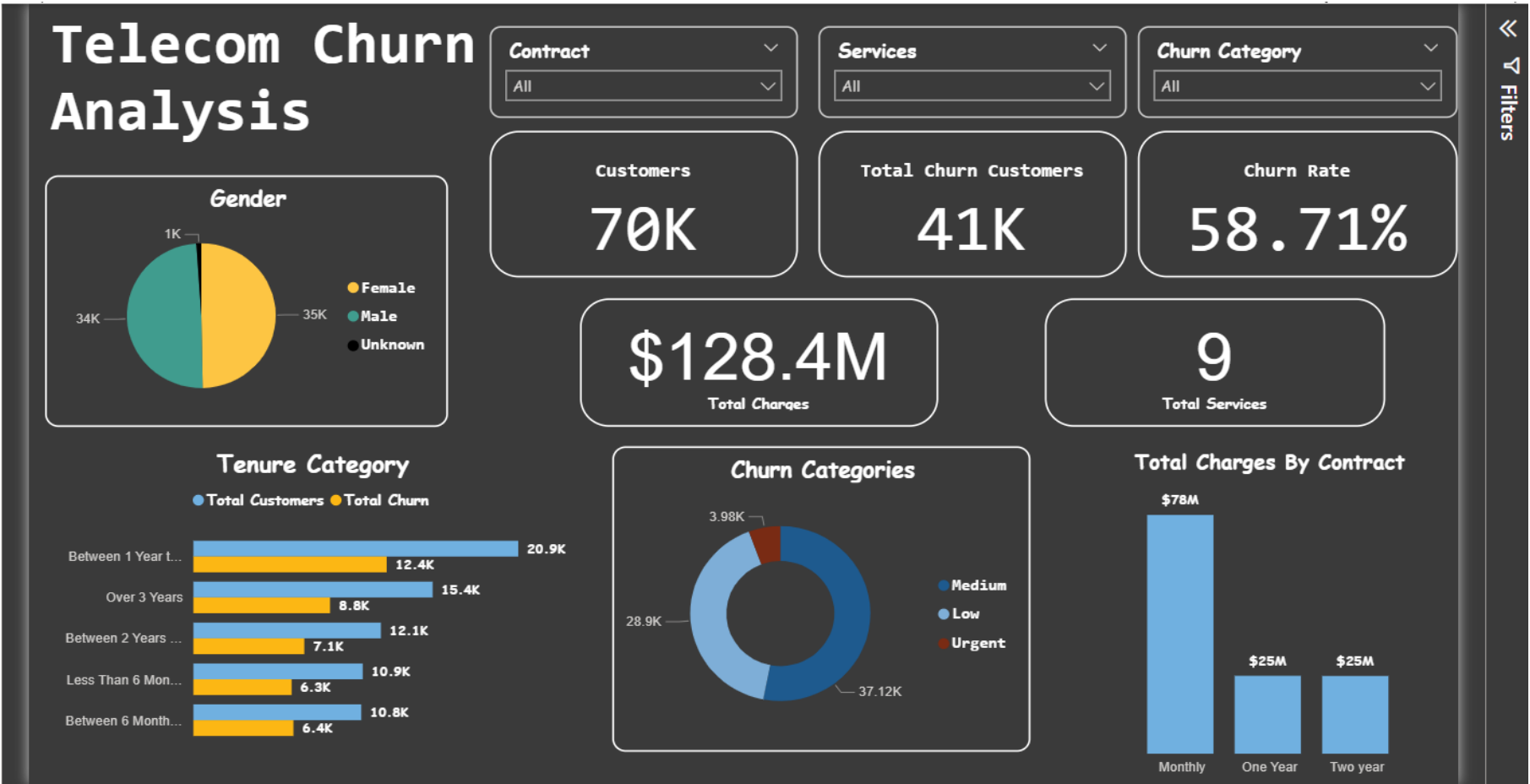
Data Modeling

Data modeling is the process of creating a visual representation of either a whole or a part of an information system to communicate connections between data points and structures. It's crucial for designing efficient databases and data warehouses.



Phase 3: Insights & Visualization with Power BI

The final phase focuses on translating complex data and model predictions into intuitive, interactive dashboards. Our goal was to empower business users with clear, actionable insights to drive customer retention strategies.



Conclusion: From Raw Data to Actionable Insights



Raw Data Input

Began with diverse, unrefined customer transaction and interaction data.



Data Transformation

Cleaned, structured, and modeled data into a robust Star Schema.



Automated Processing

Leveraged Airflow for reliable and scalable ETL and model training.



Predictive Modeling

Built a classification model to identify high-risk churn customers.



Actionable Insights

Delivered interactive Power BI dashboards for strategic decision-making.

This pipeline has significantly improved data quality, automated processing workflows, and provided invaluable business understanding, enabling proactive customer retention strategies.

Thank you